

Roomora

Technical Project Report

Room Booking Management System

Prepared by Roshitha [Gunasekaran]

Github: roshitha05

Table of Contents

1. Project Overview	3
1.1 Project Objectives	3
1.2 Scope	3
1.3 Public Project Links	3
2. Technology Stack	3
3. System Architecture	4
3.1 Application Flow	4
4. User Interface and Implementation	4
4.1 User Workflow	5
4.2 Staff Workflow	6
4.3 Demo Authentication	6
5. Browser Storage Design	6
6. Booking and Promotional Code Processing.....	7
7. Functional Testing	7
8. Deployment	8
9. Development Challenges and Solutions	8
10. Limitations and Security Considerations.....	8
11. Final Result	8
11.1 Skills Demonstrated	9
12. Future Improvements	9

1. Project Overview

Roomora is a frontend room booking management project developed to demonstrate practical web development, client-side application logic, browser-based data persistence and static deployment. The system provides separate User and Staff workflows. Users can browse available rooms, apply a promotional code and complete a reservation, while Staff can create, edit and remove room listings through a dedicated management dashboard.

1.1 Project Objectives

- Redesign the original interface into a cleaner and more consistent room booking application.
- Build separate User and Staff workflows using HTML, CSS and JavaScript.
- Allow Staff to create, update and delete room listings.
- Allow Users to browse room information and complete a booking flow.
- Persist demonstration room and booking data across browser sessions.
- Deploy the project as a publicly accessible static website using GitHub Pages.

1.2 Scope

The current version focuses on frontend behaviour and the complete client-side booking workflow rather than a production backend. It includes demo authentication, room management, room browsing, promotional-code processing, booking confirmation and browser-based persistence. A server-side API, shared database, production authentication and real payment processing are outside the current scope and are listed as possible future improvements.

1.3 Public Project Links

Live application: <https://roshitha05.github.io/roomora/>

GitHub repository: <https://github.com/roshitha05/roomora>

2. Technology Stack

Technology	Purpose
HTML5	Page structure, forms, navigation, dashboards and booking confirmation
CSS3	Responsive interface, layout, typography, cards, tables and visual states
JavaScript	Application logic, room management, booking flow and DOM updates
Web Storage API	localStorage for persistent demo data and sessionStorage for the current login session
Bootstrap 5	Modal behaviour used by the Staff room-management interface
Material Icons	Interface icons
Git / GitHub	Version control and repository hosting
GitHub Pages	Public static deployment

3. System Architecture

Roomora uses a client-side architecture. HTML files provide the application views, CSS controls presentation and responsive behaviour, and JavaScript handles user interaction and application state. The browser Web Storage API acts as the persistence layer for the portfolio version.

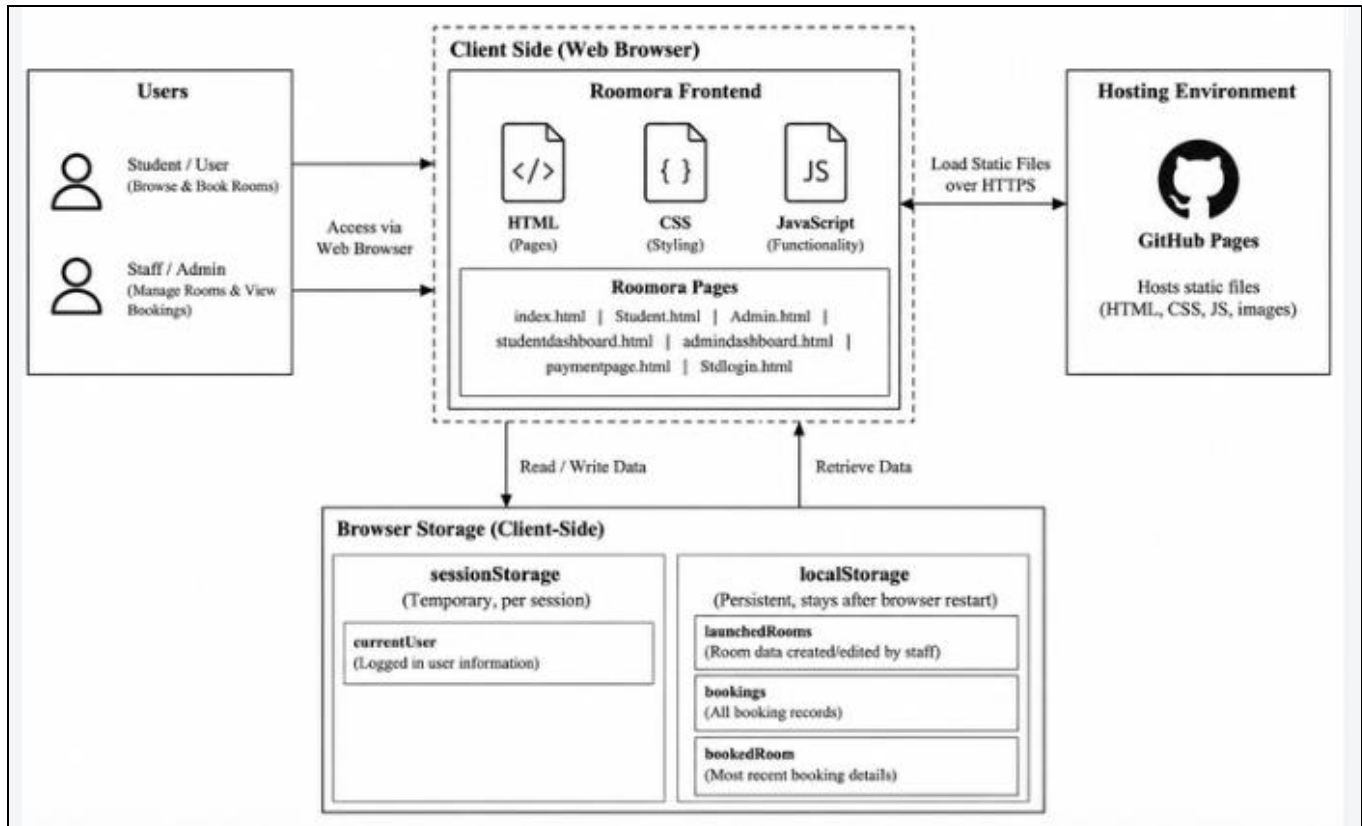


Figure 1. Roomora client-side system architecture.

3.1 Application Flow

1. A Staff user signs in and manages room records through the Staff dashboard.
2. Room records are saved to localStorage under launchedRooms.
3. The User dashboard reads the same browser-local room inventory.
4. A User selects a room and may enter the DISCOUNT10 promotional code.
5. Roomora calculates the final price and stores the completed booking data.
6. The booking confirmation page reads the latest booking and displays the reservation details.

4. User Interface and Implementation

The Roomora redesign uses a consistent modern interface across the landing page, login pages and dashboards. The visual system uses a light background, dark typography, muted blue accents, rounded cards, status pills and responsive layouts. The interface was kept intentionally lightweight so the project remains suitable for static hosting.

4.1 User Workflow

The User portal allows a visitor to sign in with the demonstration account and access the room directory. Each room displays relevant booking information such as location, capacity, date, time and price. The User can enter a promotional code and continue to the booking confirmation page after selecting a room.

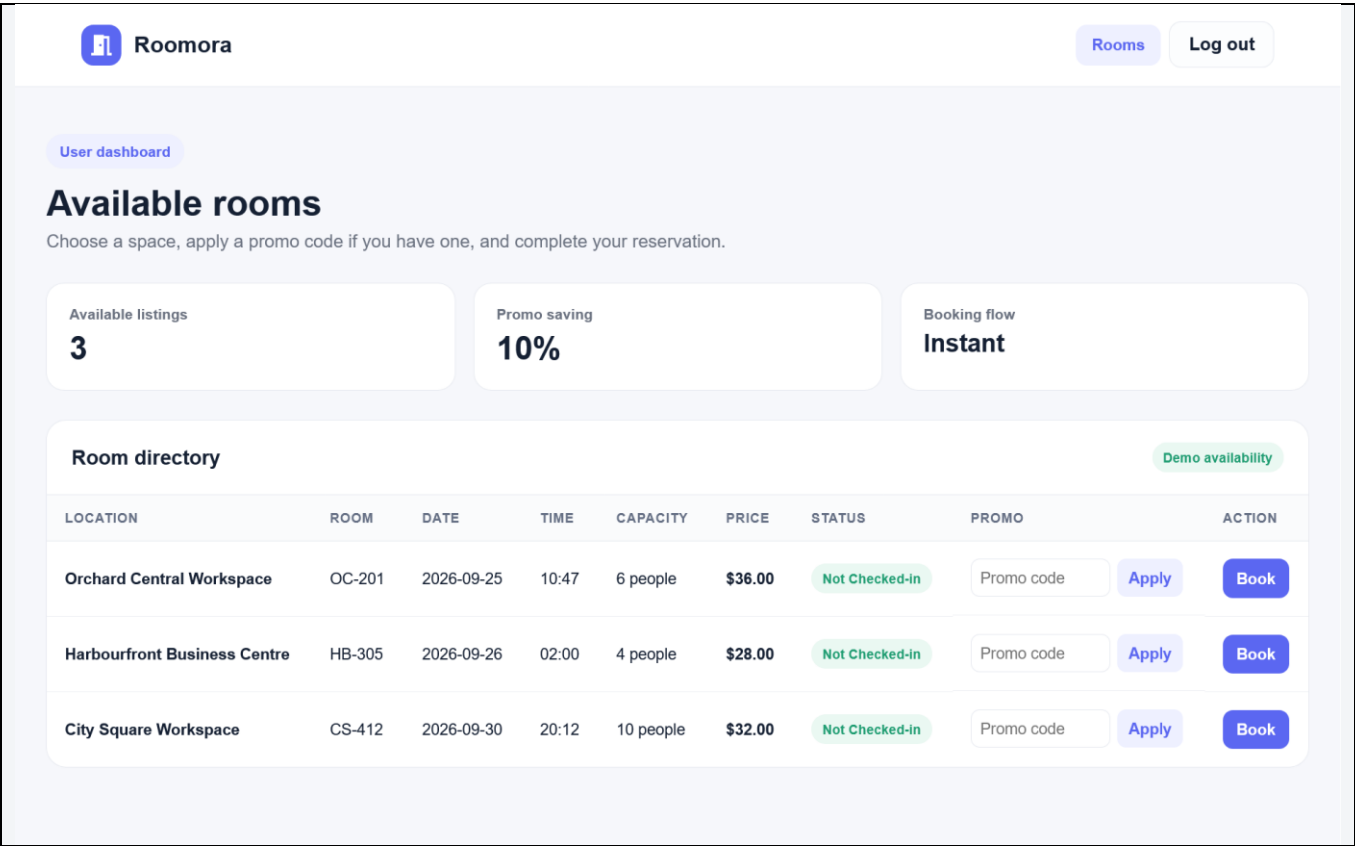


Figure 2. User dashboard displaying available rooms and booking controls.

4.2 Staff Workflow

The Staff portal provides room-management functionality. Staff can create a room record using the modal form, edit existing values and remove rooms that are no longer required. After each change, the updated room array is saved to localStorage and the inventory table is refreshed.

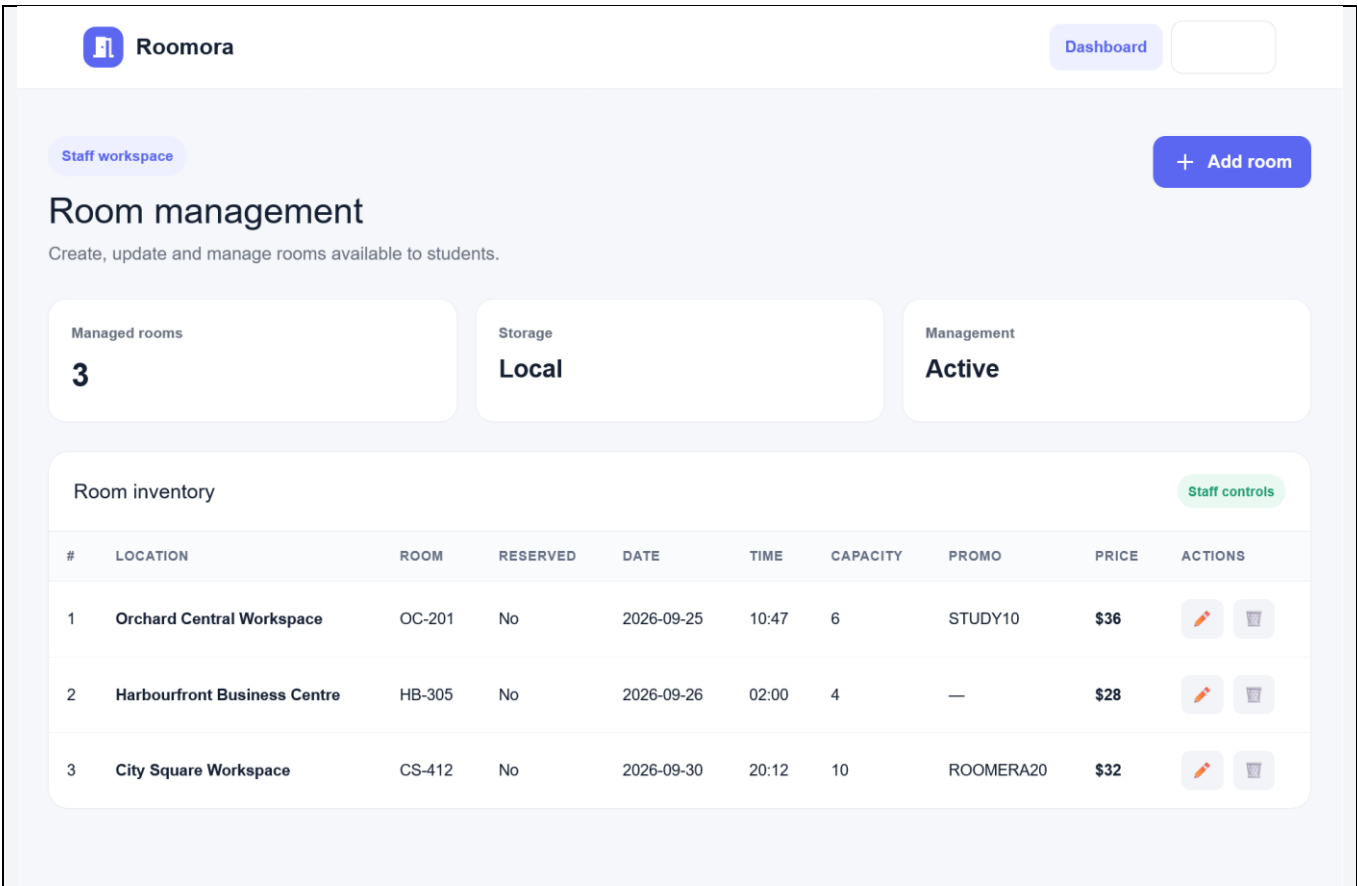


Figure 3. Staff dashboard used to manage room listings.

4.3 Demo Authentication

Portal	Username	Password
User	student	password
Staff	staff	password

The current login flow is intentionally implemented as client-side demonstration authentication. After successful login, the current account information is stored in sessionStorage. The credentials are included only so recruiters and reviewers can access both workflows without creating an account.

5. Browser Storage Design

Roomora uses the Web Storage API to keep the project functional without requiring a backend database. localStorage is used for data that should remain available after a refresh or browser restart, while sessionStorage is used for the current demonstration login session.

Storage	Key	Purpose
localStorage	launchedRooms	Stores the persistent room inventory used by Staff and User

		views
localStorage	bookings	Stores browser-local booking history
localStorage	bookedRoom	Stores the most recent booking for the confirmation page
sessionStorage	currentUser	Stores the current demonstration login session

Because localStorage is browser-specific, each visitor receives an independent copy of the demonstration data. Changes made by one visitor are not shared with another device or browser.

6. Booking and Promotional Code Processing

When a User selects a room, the booking logic retrieves the selected room data, determines the original price and checks whether a promotional code has been entered. The demonstration code DISCOUNT10 applies a 10% discount. The resulting booking object records the selected room, final price, promotional-code state and booking time before being saved to localStorage.

Input / State	Behaviour
No promo code	Booking continues at the original room price
DISCOUNT10	Applies a 10% reduction to the room price
Invalid promo code	Displays feedback and does not apply a discount
Completed booking	Stores bookedRoom and a booking-history entry before redirecting to confirmation

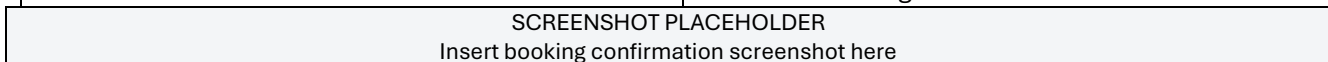


Figure 4. Booking confirmation showing reservation details and final price.

7. Functional Testing

The current project uses manual functional testing because the application is a lightweight client-side portfolio project. Testing focuses on the complete workflow, browser persistence and behaviour across the User and Staff views.

Test Area	Examples
Authentication	Valid User and Staff credentials; invalid login handling
Room management	Create, edit and delete room records
Persistence	Room records remain available after refresh
Cross-view data	Staff-created rooms appear on the User dashboard in the same browser
Promo processing	Valid, invalid and empty promotional-code states
Booking	Selected room and final price appear on confirmation
Navigation	Landing page, User portal, Staff portal and confirmation links
Responsive interface	Primary pages remain usable at narrower screen widths

8. Deployment

Roomora is deployed as a static website using GitHub Pages. The repository contains index.html at the root so GitHub Pages can use it as the site entry point. CSS and JavaScript files use relative paths so the project works correctly under the /roomora/ repository path.

Component	Deployment
Application	GitHub Pages
Source repository	GitHub
Persistence	Browser localStorage / sessionStorage
Backend service	Not used in the current version
Database	Not used in the current version

The live deployment was used to verify that the landing page and linked application pages load from the repository. Future updates pushed to the configured main branch can be published through the same GitHub Pages deployment.

9. Development Challenges and Solutions

Challenge	Resolution
Modernising the original frontend	Reworked the interface into a consistent Roomora visual system and reformatted the source for readability.
Keeping room data after the browser session ends	Moved persistent room and booking data to localStorage instead of relying only on sessionStorage.
Sharing room data between Staff and User pages	Standardised both workflows around the same launchedRooms storage key.
Updating the role terminology	Changed visible Student wording to User while retaining existing filenames to avoid unnecessary route changes.
GitHub Pages asset paths	Used relative css/ and js/ paths so files resolve correctly under the repository subpath.

10. Limitations and Security Considerations

The current implementation is intended as a frontend portfolio demonstration rather than a production booking platform. The following limitations are therefore documented explicitly:

- Authentication is client-side demo logic and can be inspected or bypassed through browser developer tools.
- Room and booking data is stored locally and is not shared between browsers, devices or users.
- There is no server-side authorisation, API validation or central database.
- The confirmation page represents a demonstration booking flow and does not process a real payment.
- Sensitive or production data should not be stored using this implementation.

11. Final Result

Roomora was completed as a working frontend portfolio project rather than a set of disconnected static pages. The final application provides a public landing page, separate User and Staff workflows, browser-

persistent room management, promotional-code processing and a booking confirmation flow. The project is structured for static deployment and can be demonstrated directly through GitHub Pages.

11.1 Skills Demonstrated

- Frontend development with HTML, CSS and JavaScript
- Responsive interface design and reusable visual styling
- JavaScript DOM manipulation and event handling
- Client-side CRUD-style room management
- Web Storage API and browser-based state persistence
- Form handling and promotional-code validation
- Multi-page application navigation
- Static deployment using GitHub Pages
- Git and GitHub version-control workflow
- Technical documentation and clear communication of implementation limitations

12. Future Improvements

The current scope is intentionally focused on the frontend booking workflow. If the project is extended, the following improvements would add useful functionality while moving the system closer to a production architecture:

- Replace demo authentication with secure backend authentication.
- Store rooms and bookings in a shared database.
- Add server-side validation and role-based authorisation.
- Implement booking-conflict detection and availability rules.
- Add room search and filtering.
- Add a User booking-history interface.
- Add automated JavaScript tests for booking and room-management logic.